REPUBLIC OF CAMEROON
Peace - work - Fatherland
MINISTRY OF HIGHER EDUCATION

REGIONAL DELEGATION OF CENTER

PKFokam Institute of Excellence

Dignity - Faith – Responsibility
P.O.BOX 11646 Yaoundé – Cameroon

RÉPUBLIQUE DU CAMEROUN
Paix - travail – Patrie
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR

DÉLÉGATION REGIONALE DU CENTRE

PKFokam Institute of Excellence

Dignité - Foi – Responsabilité
P.O. Box 11646 Yaoundé – Cameroun

Project Progress Report

WIGMS (WEDDING INVITATION & GIFT MANAGEMENT SYSTEM)

Course	C Programming Academic Year 2025-2026
Institution	PKFIE Pkfokam Institute of Excellence
Lecturer	Mbietieu Amos Mb.
Report Type	Module Implementation Report
Files Covered	types.h | ui.h | ui.c
Language	C (Modular Architecture)
Date	13 March 2026

Group Members	Participation
Pettang Feumba Roddy Alann	20%
Tiofack Kaze Franck	20%
Mep Meku Audrey	20%
Tamo Taboum Israel	20%
Djatche Raziel Steevi	20%

LECTURER: MR. MBIETEU AMOS MB.

DATE: FRIDAY, 3TH APRIL 2026

Introduction

This is the second weekly progress report for the WIGMS (Wedding Invitation and Gift Management System) project. Report 1 documented the GitHub repository setup, the shared types.h library, Module 1 (Person Management), and Module 2 (Category Management). This report continues from where Report 1 ended and documents the implementation of Module 3 (Priority Management DPR Merge Sort), Module 4 (Gift Management with file persistence), and Module 5 (Graphical User Interface using the GTK+ 3.0 library). It also presents the brainstorming and initial implementation of three additional modules proposed to extend the system: Module 6 (Parking Management), Module 7 (Meal and Table Seating), and Module 8 (Event Timeline and Schedule).

3. Module 3; Priority Management (DPR Merge Sort)

3.1 Objectives

The main objectives of Module 3 are:

- Implement the DPR (Divide-Process-Recombine) algorithmic paradigm on the category linked list
- Sort guests within each category by age using insertion sort on the small fixed-size array
- Sort guests by social class priority (VIP first, then Family, then Friend, then Other)
- Apply Merge Sort to the category linked list itself, ordering categories by guest count

3.2 What Is DPR?

DPR stands for Divide – Process – Recombine. It is the academic name given in this project to the Merge Sort algorithm. The algorithm operates in three recursive steps:

Step	Action	Function
DIVIDE	Find the middle of the linked list and split it into two halves	get_middle() using Floyd tortoise & hare
PROCESS	Recursively sort both halves independently	merge_sort_internal() calling itself
RECOMBINE	Merge the two sorted halves into one sorted list	merge() comparing heads one by one

3.3 Why Merge Sort for a Linked List?

Merge Sort is the ideal algorithm for linked lists because it does not require random access by index (which arrays have but linked lists do not). It achieves O(n log n) time complexity in all cases and is stable, meaning equal elements preserve their original relative order. Bubble sort, used in Module 2 for a simpler sort, has $O(n^2)$ complexity and would be too slow for large guest lists.

3.4 File Structure

File	Role
include/priority.h	Header function declarations and include guard
src/priority.c	Implementation all DPR functions

3.5 Key Functions

3.5.1 get_middle() Floyd's Tortoise & Hare

This function finds the middle node of the linked list in a single pass using two pointers: slow moves one step at a time and fast moves two steps at a time. When fast reaches the end, slow is at the midpoint. This avoids counting the list length first, saving an extra traversal.

3.5.2 merge() The Recombine Step

The merge function receives two already-sorted sub-lists and produces one combined sorted list. It compares the heads of each sub-list and attaches the smaller one to the result, then recurses with the remaining nodes. When one sub-list is exhausted, the remaining nodes of the other are appended directly.

3.5.3 prioritize_by_age() and prioritize_by_social_class()

These functions sort the guests[] array inside each Category node. Because the array holds at most 4 elements, insertion sort is used it is faster than merge sort for very small arrays due to lower overhead. The class_rank() helper maps social class strings to integer priority values: VIP=1, Family=2, Friend=3, Other=4.

3.6 Step-by-Step Implementation Procedure

STEP 1 Create priority.h

Create include/priority.h. Add the include guard (#ifndef PRIORITY_H / #define PRIORITY_H / #endif). Declare the three public functions: priority_sort_by_age(), priority_sort_by_class(), priority_merge_sort_categories(). Mark internal helpers as static in priority.c so they are invisible outside the file.

STEP 2 Implement get_middle()

In src/priority.c, implement get_middle() with the slow/fast pointer technique described above. Test with a manually built 5-node list and verify slow lands on node 3.

STEP 3 Implement merge()

Implement the merge() function with the by_age flag controlling which field is used for comparison. Test by creating two pre-sorted 3-node sub-lists and verifying the merged result is in order.

STEP 4 Implement merge_sort_internal()

Implement the recursive DPR function: find middle, set mid->next = NULL to cut the list, recursively sort both halves, merge results. Test with a fully unsorted 6-node list.

STEP 5 Implement insertion sort for guests

Implement priority_sort_by_age() and priority_sort_by_class() using insertion sort on the fixed guests[4] array inside each node.

STEP 6 Commit to GitHub

git add include/priority.h src/priority.c && git commit -m 'Module 3: DPR Merge Sort complete' && git push origin module3-priority

3.7 GitHub Contribution

```
git checkout -b module3-priority
git add include/priority.h src/priority.c
git commit -m "Module 3: implemented DPR Merge Sort sort by age, class, guest count"
git push origin module3-priority
```

priority.c priority.h

```c
#ifndef PRIORITY_H
#define PRIORITY_H

#include "types.h"

/* ============================================================
   MODULE 3 - Priority Management (DPR / Merge Sort)
   ============================================================ */

/* Console versions */
void prioritize_by_age(Category *tete);
void prioritize_by_social_class(Category *tete);
void merge_sort_categories(Category **tete);

/* GUI-friendly wrappers (same behavior, different names) */
void priority_sort_by_age(Category *tete);
void priority_sort_by_class(Category *tete);
void priority_merge_sort_categories(Category **tete);

#endif /* PRIORITY_H */
```

priority.c priority.h

```c
#include <stdio.h>
#include <string.h>

#include "priority.h"

static Category* get_middle(Category *head) {
    if (!head) return NULL;

    Category *slow = head;
    Category *fast = head->next;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

static Category* merge(Category *left, Category *right, int by_age) {
    if (left == NULL) return right;
    if (right == NULL) return left;

    int left_val;
    int right_val;

    if (by_age) {
        left_val = (left->guest_count > 0) ? left->guests[0].age : 0;
        right_val = (right->guest_count > 0) ? right->guests[0].age : 0;
    } else {
        left_val = left->guest_count;
        right_val = right->guest_count;
    }

    if (left_val <= right_val) {
        left->next = merge(left->next, right, by_age);
        return left;
    } else {
        right->next = merge(left, right->next, by_age);
        return right;
    }
}

static Category* merge_sort_internal(Category *head, int by_age) {
    if (!head || !head->next) return head;

    Category *mid = get_middle(head);
    Category *right = mid->next;
    mid->next = NULL;

    Category *left_sorted  = merge_sort_internal(head, by_age);
    Category *right_sorted = merge_sort_internal(right, by_age);

    return merge(left_sorted, right_sorted, by_age);
}

void prioritize_by_age(Category *tete) {
    Category *curr = tete;
    while (curr != NULL) {
        for (int i = 1; i < curr->guest_count; i++) {
            Person key = curr->guests[i];
            int j = i - 1;
            while (j >= 0 && curr->guests[j].age > key.age) {
                curr->guests[j + 1] = curr->guests[j];
                j--;
            }
            curr->guests[j + 1] = key;
        }
        curr = curr->next;
    }
    printf("  Guests within each category sorted by age (ascending).\n");
}

static int class_rank(const char *cls) {
    if (!cls) return 4;
    if (strncmp(cls, "VIP", 3) == 0) return 1;
    if (strncmp(cls, "Family", 6) == 0) return 2;
    if (strncmp(cls, "Friend", 6) == 0) return 3;
    return 4;
}

void prioritize_by_social_class(Category *tete) {
    Category *curr = tete;
    while (curr != NULL) {
        for (int i = 1; i < curr->guest_count; i++) {
            Person key = curr->guests[i];
            int j = i - 1;
            while (j >= 0 &&
                   class_rank(curr->guests[j].social_class) >
                   class_rank(key.social_class)) {
                curr->guests[j + 1] = curr->guests[j];
                j--;
            }
            curr->guests[j + 1] = key;
        }
        curr = curr->next;
    }
    printf("  Guests sorted by social class (VIP first).\n");
}

void merge_sort_categories(Category **tete) {
    if (!tete) return;
    *tete = merge_sort_internal(*tete, 0 /* by guest_count */);
    printf("  Categories sorted by guest count using Merge Sort (DPR).\n");
}

void priority_sort_by_age(Category *tete) {
    prioritize_by_age(tete);
}

void priority_sort_by_class(Category *tete) {
    prioritize_by_social_class(tete);
}

void priority_merge_sort_categories(Category **tete) {
    merge_sort_categories(tete);
}
```

4. Module 4: Gift Management

4.1 Objectives

- Register gifts linked to specific guests by guest_id
- Display all gifts with their monetary value in FCFA
- Update and delete gifts by gift_id
- Calculate the total gift value for the entire wedding
- Save gifts to a binary file on disk so data survives program restarts
- Load gifts automatically from the binary file at startup

4.2 Data Structure

Module 4 uses a fixed-size array of Gift structs rather than a linked list. This choice is appropriate because gifts are frequently accessed by index, the maximum count is bounded (MAX_GIFTS = 200), and the flat array structure makes binary file I/O trivially simple one fwrite call writes all gifts at once.

```
typedef struct {
 int gift_id;
 char name[100];
 float value; /* in FCFA */
 int guest_id; /* foreign key links to a guest */
} Gift;

#define MAX_GIFTS 200
```

4.3 File Persistence, Binary I/O

The most technically distinctive feature of Module 4 is binary file persistence. When the program exits, all RAM is cleared. To keep gift data between sessions, the gifts array is written to a .dat file using fwrite() and read back with fread() at the next startup.

Function	Mode	What It Does
gift_save()	fopen("wb") write binary	Writes the integer count as a header, then writes all Gift structs in one fwrite() call
gift_load()	fopen("rb") read binary	Reads the count header first, then reads exactly that many Gift structs back into the array

4.4 Array Delete Shift Pattern

Because arrays do not support true deletion, removing a gift at index i requires shifting every element after it one position to the left, then decrementing the count. The last element is effectively orphaned beyond the active range and will be overwritten by the next insertion.

4.5 Step-by-Step Implementation Procedure

STEP 1 Create gift.h
Write include/gift.h with the Gift struct, MAX_GIFTS macro, extern gift_id_counter, and all function declarations.

STEP 2 Implement CRUD functions

In src/gift.c implement gift_register() (array append with bounds check), gift_delete() (array shift), gift_update() (linear search then modify), and gift_total_value() (accumulator loop).

STEP 3 Implement file I/O

Implement gift_save() and gift_load() exactly as shown above. Test by saving 3 gifts, closing the program in Dev-C++, running again, and verifying the 3 gifts are reloaded.

STEP 4 Integrate in main.c

Call gift_load() at program startup before the menu loop. Call gift_save() before program exit.

STEP 5 Test persistence

Add 5 gifts, close the program, reopen, verify all 5 reload correctly and gift IDs continue from where they left off.

STEP 6 Commit to GitHub

git add include/gift.h src/gift.c && git commit -m 'Module 4: Gift CRUD + binary file persistence' && git push origin module4-gifts

priority.c | priority.h | gift.h | gift.c

```c
#ifndef GIFT_H
#define GIFT_H

#include "types.h"

/* ========================================================
   MODULE 4 - Gift Management (Array + File Persistence)
   ======================================================== */

/* Console-interactive versions */
void  register_gift(Gift gifts[], int *count);
void  display_gifts(Gift gifts[], int count);
void  update_gifts(Gift gifts[], int count, int gift_id);
void  delete_gift(Gift gifts[], int *count, int gift_id);
float total_gift_value(Gift gifts[], int count);
void  save_gifts(Gift gifts[], int count, const char *filename);
void  load_gifts(Gift gifts[], int *count, const char *filename);

extern int gift_id_counter;

/* ========================================================
   GUI-friendly wrappers (data-driven)
   ======================================================== */
void  gift_register(Gift gifts[], int *count,
                    const char *name, float value, int guest_id);
void  gift_display(Gift gifts[], int count); /* optional, not required */
void  gift_update(Gift gifts[], int count, int gift_id,
                    const char *name, float value, int guest_id);
void  gift_delete(Gift gifts[], int *count, int gift_id);
float gift_total_value(Gift gifts[], int count);
void  gift_save(Gift gifts[], int count, const char *filename);
void  gift_load(Gift gifts[], int *count, const char *filename);

#endif /* GIFT_H */
```

gift.c 2

C: > Users > hp > Desktop > PROJECT 1 > Wedding Test > src > gift.c > display_gifts(Gift [], int)

```c
#include <stdio.h>Use Ctrl + I to generate code
#include <string.h>
#include <errno.h>

#ifdef _WIN32
  #include <direct.h>
#else
  #include <sys/stat.h>
#endif

#include "gift.h"
#include "person.h" /* for clear_input_buffer() */

int gift_id_counter = 1;

Windsurf: Refactor | Explain | Generate Function Comment |
static void ensure_data_dir(void) {
#ifdef _WIN32
    /* Best-effort: ignore errors if it already exists */
    _mkdir("data");
#else
    mkdir("data", 0755);
#endif
}

/* ---------------------------------------------------------
   register_gift() - console-interactive
   --------------------------------------------------------- */
Windsurf: Refactor | Explain |
void register_gift(Gift gifts[], int *count) {
    if (!gifts || !count) return;
    if (*count >= MAX_GIFTS) {
        printf("  Gift registry is full (%d gifts).\n", MAX_GIFTS);
        return;
    }

    Gift g;
    g.gift_id = gift_id_counter++;

    printf("  Gift name      : ");
    fgets(g.name, sizeof(g.name), stdin);
    g.name[strcspn(g.name, "\n")] = '\0';

    printf("  Gift value (FCFA): ");
    scanf("%f", &g.value);
    clear_input_buffer();

    printf("  Guest ID who gave this gift: ");
    scanf("%d", &g.guest_id);
    clear_input_buffer();

    gifts[*count] = g;
    (*count)++;

    printf("  Gift '%s' registered (ID: %d).\n", g.name, g.gift_id);
}

Windsurf: Refactor | Explain | Generate Function Comment |
void display_gifts(Gift gifts[], int count) {
    if (!gifts || count <= 0) {
        printf("  No gifts registered yet.\n");
        return;
    }

    printf("\n  %-5s %-25s %-12s %-10s\n",
           "ID", "Gift Name", "Value(FCFA)", "Guest ID");
    printf("  %-5s %-25s %-12s %-10s\n",
           "---", "-------------------------", "----------", "--------");

    float total = 0.0f;
    for (int i = 0; i < count; i++) {
        printf("  %-5d %-25s %-12.0f %-10d\n",
               gifts[i].gift_id,
```

gift.c 2

C: > Users > hp > Desktop > PROJECT 1 > Wedding Test > src > gift.c > ...

```c
void display_gifts(Gift gifts[], int count) {
    for (int i = 0; i < count; i++) {
        total += gifts[i].value;
    }
    printf("  ---------------------------------------------\n");
    printf("  TOTAL VALUE: %.0f FCFA  (%d gifts)\n", total, count);
}

Windsurf: Refactor | Explain | Generate Function Comment |
void update_gifts(Gift gifts[], int count, int gift_id) {
    if (!gifts) return;

    int idx = -1;
    for (int i = 0; i < count; i++) {
        if (gifts[i].gift_id == gift_id) {
            idx = i;
            break;
        }
    }
    if (idx == -1) {
        printf("  Gift ID %d not found.\n", gift_id);
        return;
    }

    char buf[100];
    printf("  New name (Enter to keep [%s]): ", gifts[idx].name);
    fgets(buf, sizeof(buf), stdin);
    buf[strcspn(buf, "\n")] = '\0';
    if (strlen(buf) > 0) {
        strncpy(gifts[idx].name, buf, sizeof(gifts[idx].name) - 1);
        gifts[idx].name[sizeof(gifts[idx].name) - 1] = '\0';
    }

    printf("  New value (0 to keep [%.0f]): ", gifts[idx].value);
    float v = 0.0f;
    scanf("%f", &v);
    clear_input_buffer();
    if (v != 0.0f) gifts[idx].value = v;

    printf("  Gift updated.\n");
}

Windsurf: Refactor | Explain | Generate Function Comment |
void delete_gift(Gift gifts[], int *count, int gift_id) {
    if (!gifts || !count) return;

    int idx = -1;
    for (int i = 0; i < *count; i++) {
        if (gifts[i].gift_id == gift_id) {
            idx = i;
            break;
        }
    }
    if (idx == -1) {
        printf("  Gift ID %d not found.\n", gift_id);
        return;
    }

    for (int i = idx; i < *count - 1; i++) {
        gifts[i] = gifts[i + 1];
    }
    (*count)--;
    printf("  Gift ID %d deleted.\n", gift_id);
}

Windsurf: Refactor | Explain | Generate Function Comment |
float total_gift_value(Gift gifts[], int count) {
    float total = 0.0f;
    for (int i = 0; i < count; i++) {
        total += gifts[i].value;
    }
    return total;
}
```

gift.c 2

C: > Users > hp > Desktop > PROJECT 1 > Wedding Test > src > C gift.c > ...

```c
void load_gifts(Gift gifts[], int *count, const char *filename) {
    if (read > 0) {
        gift_id_counter = gifts[read - 1].gift_id + 1;
    } else {
        gift_id_counter = 1;
    }
}

/* ========================================================
   GUI-friendly wrappers expected by ui_gtk.c
   ========================================================== */

Windsurf: Refactor | Explain | Generate Function Comment | ×
void gift_register(Gift gifts[], int *count,
                   const char *name, float value, int guest_id) {
    if (!gifts || !count) return;
    if (*count >= MAX_GIFTS) return;

    Gift g;
    g.gift_id = gift_id_counter++;
    strncpy(g.name, name ? name : "Gift", sizeof(g.name) - 1);
    g.name[sizeof(g.name) - 1] = '\0';
    g.value = value;
    g.guest_id = guest_id;

    gifts[*count] = g;
    (*count)++;
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void gift_display(Gift gifts[], int count) {
    display_gifts(gifts, count);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void gift_update(Gift gifts[], int count, int gift_id,
                 const char *name, float value, int guest_id) {
    if (!gifts) return;
    int idx = -1;
    for (int i = 0; i < count; i++) {
        if (gifts[i].gift_id == gift_id) {
            idx = i;
            break;
        }
    }
    if (idx == -1) return;

    if (name && name[0]) {
        strncpy(gifts[idx].name, name, sizeof(gifts[idx].name) - 1);
        gifts[idx].name[sizeof(gifts[idx].name) - 1] = '\0';
    }
    gifts[idx].value = value;
    gifts[idx].guest_id = guest_id;
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void gift_delete(Gift gifts[], int *count, int gift_id) {
    delete_gift(gifts, count, gift_id);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
float gift_total_value(Gift gifts[], int count) {
    return total_gift_value(gifts, count);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void gift_save(Gift gifts[], int count, const char *filename) {
    save_gifts(gifts, count, filename);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void gift_load(Gift gifts[], int *count, const char *filename) {
    load_gifts(gifts, count, filename);
}
```

5. Module 5: Graphical User Interface (GTK+ 3.0)

5.1 Objectives

Module 5 replaces the original console menu system with a complete graphical user interface built using the GTK+ 3.0 library. The interface provides a professional windowed application with five tabs, interactive data tables, forms, modal dialogs, CSS theming, and a live statistics dashboard. It acts as the Controller layer of the MVC architecture it contains no business logic, only routing user actions to the correct module functions.

5.2 What Is GTK?

GTK (GIMP Toolkit) is a professional open-source C library for building graphical user interfaces with real windows, buttons, forms, menus, and dialogs. It is the same library used by the GNOME desktop environment and applications like GIMP and Inkscape. It was chosen for this project because it integrates directly with C, requires no additional language, and is available on Linux via a simple apt install command.

> Install command: sudo apt install libgtk-3-dev pkg-config Compile flag: $(shell pkg-config --cflags --libs gtk+-3.0)

5.3 GTK Architecture The Widget Tree

GTK builds a GUI as a tree of widgets. Every element visible on screen a window, a button, a text field, a table is a widget. Widgets are nested inside containers (boxes, grids, notebooks). The entire tree is made visible with one call to gtk_widget_show_all().

```
GtkWindow (main application window)
  └─ GtkNotebook (5 tabs)
  ├─ Tab 1: Categories → GtkBox → GtkTreeView + GtkGrid form
  ├─ Tab 2: Persons → GtkBox → GtkTreeView + GtkGrid form
  ├─ Tab 3: Priority → GtkBox → buttons + GtkTreeView
  ├─ Tab 4: Gifts → GtkBox → GtkTreeView + GtkGrid form
  └─ Tab 5: Dashboard → GtkBox → stat card GtkFrames
```

5.4 Signal-Callback Event Model

GTK is event-driven. After calling gtk_main(), the program waits in an event loop. When the user clicks a button or selects a row, GTK fires a signal. The g_signal_connect() function wires a signal to a callback function written in C.

5.5 Key GTK Widgets Used

Widget	Purpose in WIGMS
GtkWindow	Top-level application window root of the widget tree
GtkHeaderBar	Modern title bar with embedded subtitle
GtkNotebook	5-tab panel container
GtkBox	Linear container for vertical and horizontal layouts

Widget	Purpose in WIGMS
GtkGrid	2-D layout for all data entry forms
GtkTreeView + GtkListStore	Spreadsheet-style data table with live model
GtkEntry	Single-line text input names, codes, plates
GtkSpinButton	Numeric up/down spinner age, value, count
GtkComboBoxText	Drop-down list Side, social class, vehicle type
GtkButton	Clickable button Add, Update, Delete, Sort, Save
GtkLabel	Static and dynamic text titles, status, totals
GtkMessageDialog	Pre-built modal dialog for info, errors, and confirmations
GtkCssProvider	Loads CSS rules for the navy/orange colour theme
GtkScrolledWindow	Adds scroll bars around TreeView tables
GtkFrame	Labelled border box used for Dashboard stat cards

5.6 CSS Theming

GTK 3 supports CSS styling identical in syntax to web CSS. A CSS string is loaded at application startup and applied globally to the screen via GtkCssProvider. This gives all buttons, labels, trees, and stat cards a consistent professional colour scheme using the project's navy and orange palette.

5.7 AppState Shared Application Data

All GTK tabs share a single AppState struct declared in ui_gtk.h. A pointer to this struct is passed as user_data to every callback. This ensures that changes made on one tab (such as adding a category) are immediately visible on all other tabs.

5.8 Step-by-Step Implementation Procedure

STEP 1 Install GTK

Run: sudo apt install libgtk-3-dev pkg-config. Verify: pkg-config --modversion gtk+-3.0 should print 3.x.x

STEP 2 Update Makefile

Add $(shell pkg-config --cflags gtk+-3.0) to CFLAGS and $(shell pkg-config --libs gtk+-3.0) to LIBS in the Makefile.

STEP 3 Create ui_gtk.h

Write the AppState struct, declare ui_gtk_run() as the entry point, and declare all shared helper functions (dialogs, CSS).

STEP 4 Implement ui_apply_css()

Write the CSS string as a C string literal and load it with gtk_css_provider_load_from_data(). Apply globally with gtk_style_context_add_provider_for_screen().

STEP 5 Build the main window

In ui_gtk_run(): call gtk_init(), create GtkWindow, attach GtkHeaderBar, create GtkNotebook, call each build_*_tab() function, append each returned widget as a tab page.

STEP 6 Build each tab

Implement build_category_tab(), build_person_tab(), build_priority_tab(), build_gift_tab(), and build_stats_tab(). Each returns a GtkWidget* root container.

STEP 7 Connect destroy signal

g_signal_connect(win, 'destroy', G_CALLBACK(on_main_destroy), state) this auto-saves gifts and frees the category list when the window is closed.

STEP 8 Compile and test

Run make. Run ./wigms. Verify all 5 tabs open, buttons respond, and data displays correctly in the TreeView tables.

ui_gtk.h

```c
#ifndef UI_GTK_H
#define UI_GTK_H

#include <gtk/gtk.h>
#include "types.h"
#include "parking.h"
#include "seating.h"
#include "schedule.h"

/* ============================================================
   MODULE 5 - GTK Graphical User Interface
   ui_gtk.h : public declarations
   ============================================================ */

typedef struct {
    Category *cat_head;          /* head of category linked list */
    Gift      gifts[MAX_GIFTS];  /* flat gift array */
    int       gift_count;        /* number of active gifts */

    /* Modules 6-8 */
    ParkingLot      parking;
    DiningHall      dining;
    WeddingSchedule schedule;
} AppState;

/* Entry point for the GTK GUI */
void ui_gtk_run(int *argc, char ***argv, AppState *state);

/* Shared dialog helpers */
void     ui_apply_css(void);
void     ui_show_info_dialog(GtkWindow *parent, const char *title, const char *msg);
void     ui_show_error_dialog(GtkWindow *parent, const char *title, const char *msg);
gboolean ui_confirm_dialog(GtkWindow *parent, const char *question);

#endif /* UI_GTK_H */
```

ui_gtk.c 2

C: > Users > hp > Desktop > PROJECT 1 > Wedding Test > src > ui_gtk.c > ...

```c
#include <gtk/gtk.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>

#include "ui_gtk.h"
#include "category.h"
#include "person.h"
#include "priority.h"
#include "gift.h"
#include "parking.h"
#include "seating.h"
#include "schedule.h"

#define GIFTS_FILE "data/gifts.dat"
#define PARKING_FILE "data/parking.dat"
#define SEATING_FILE "data/seating.dat"
#define SCHEDULE_FILE "data/schedule.dat"
#define APP_TITLE  "WIGMS - Wedding Invitation & Gift Management"
#define APP_W      1100
#define APP_H      720

/* ========================================================
   Per-tab contexts
   ======================================================== */

typedef struct {
    AppState      *state;
    GtkWidget     *main_win;
    GtkListStore  *store;
    GtkTreeView   *tree;
    GtkEntry      *entry_code;
    GtkSpinButton *spin_guests;
    GtkEntry      *g_name[4];
    GtkSpinButton *g_age[4];
    GtkEntry      *g_class[4];
    GtkComboBoxText *g_side[4];
    int            selected_id; /* -1 = none */
} CatCtx;

typedef struct {
    AppState      *state;
    GtkWidget     *main_win;
    GtkListStore  *store;
    GtkTreeView   *tree;
    GtkEntry      *entry_name;
    GtkSpinButton *spin_age;
    GtkEntry      *entry_class;
    GtkComboBoxText *combo_side;
    int            selected_cat_id;
    int            selected_guest_idx;
} PersonCtx;

typedef struct {
    AppState     *state;
    GtkWidget    *main_win;
    GtkListStore *store;
    GtkTreeView  *tree;
    GtkLabel     *lbl_status;
} PrioCtx;

typedef struct {
    AppState      *state;
    GtkWidget     *main_win;
    GtkListStore  *store;
    GtkTreeView   *tree;
    GtkEntry      *entry_name;
    GtkSpinButton *spin_value;
    GtkSpinButton *spin_guest;
    GtkLabel      *lbl_total;
    int            selected_id;
} GiftCtx;
```

```c

typedef struct {
    AppState *state;
    GtkLabel *lbl_cats;
    GtkLabel *lbl_guests;
    GtkLabel *lbl_gifts;
    GtkLabel *lbl_total;
} StatsCtx;

typedef struct {
    AppState           *state;
    GtkWidget          *main_win;
    GtkListStore       *store;
    GtkTreeView        *tree;
    GtkComboBoxText    *combo_zone_filter;
    GtkSpinButton      *spin_guest;
    GtkEntry           *entry_plate;
    GtkComboBoxText    *combo_vehicle;
    GtkComboBoxText    *combo_preferred_zone;
    GtkLabel           *lbl_stats;
    int                 selected_spot_id;
} ParkCtx;

typedef struct {
    AppState        *state;
    GtkWidget       *main_win;
    GtkListStore    *store;
    GtkTreeView     *tree;
    GtkEntry        *entry_table_name;
    GtkSpinButton   *spin_table_capacity;
    GtkComboBoxText *combo_table_type;
    GtkSpinButton   *spin_assign_table;
    GtkSpinButton   *spin_assign_guest;
    GtkEntry        *entry_assign_name;
    GtkComboBoxText *combo_meal;
    GtkComboBoxText *combo_diet;
    GtkSpinButton   *spin_move_guest;
    GtkSpinButton   *spin_move_table;
    GtkSpinButton   *spin_rsvp_guest;
    GtkComboBoxText *combo_rsvp;
    GtkLabel        *lbl_catering;
} SeatCtx;

typedef struct {
    AppState        *state;
    GtkWidget       *main_win;
    GtkListStore    *store;
    GtkTreeView     *tree;
    GtkEntry        *entry_title;
    GtkComboBoxText *combo_category;
    GtkComboBoxText *combo_priority;
    GtkSpinButton   *spin_start_h;
    GtkSpinButton   *spin_start_m;
    GtkSpinButton   *spin_dur;
    GtkEntry        *entry_resp;
    GtkEntry        *entry_loc;
    GtkSpinButton   *spin_delay;
    GtkLabel        *lbl_countdown;
    int              selected_event_id;
} SchedCtx;

/* Single instance contexts so callbacks can refresh other tabs */
static CatCtx    *g_cat_ctx    = NULL;
static PersonCtx *g_person_ctx = NULL;
static PrioCtx    *g_prio_ctx   = NULL;
static GiftCtx   *g_gift_ctx   = NULL;
static StatsCtx  *g_stats_ctx  = NULL;
static ParkCtx   *g_park_ctx   = NULL;
static SeatCtx   *g_seat_ctx   = NULL;
static SchedCtx  *g_sched_ctx  = NULL;

```

```c
    category_free_all(&state->cat_head);

    if (g_cat_ctx) g_free(g_cat_ctx);
    if (g_person_ctx) g_free(g_person_ctx);
    if (g_prio_ctx) g_free(g_prio_ctx);
    if (g_gift_ctx) g_free(g_gift_ctx);
    if (g_park_ctx) g_free(g_park_ctx);
    if (g_seat_ctx) g_free(g_seat_ctx);
    if (g_sched_ctx) g_free(g_sched_ctx);
    if (g_stats_ctx) g_free(g_stats_ctx);

    gtk_main_quit();
}

/* ========================================================
   GTK entry point
   ======================================================== */

Windsurf: Refactor | Explain | Generate Function Comment | ×
void ui_gtk_run(int *argc, char ***argv, AppState *state) {
    gtk_init(argc, argv);
    ui_apply_css();

    /* Load persisted gifts before building Gift tab */
    gift_load(state->gifts, &state->gift_count, GIFTS_FILE);

    GtkWidget *win = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_title(GTK_WINDOW(win), APP_TITLE);
    gtk_window_set_default_size(GTK_WINDOW(win), APP_W, APP_H);
    gtk_window_set_position(GTK_WINDOW(win), GTK_WIN_POS_CENTER);
    g_signal_connect(win, "destroy", G_CALLBACK(on_main_destroy), state);

    GtkWidget *hbar = gtk_header_bar_new();
    gtk_header_bar_set_title(GTK_HEADER_BAR(hbar), "WIGMS");
    gtk_header_bar_set_subtitle(GTK_HEADER_BAR(hbar), "Wedding Invitation & Gift Mana
    gtk_header_bar_set_show_close_button(GTK_HEADER_BAR(hbar), TRUE);
    gtk_window_set_titlebar(GTK_WINDOW(win), hbar);

    GtkWidget *nb = gtk_notebook_new();
    gtk_notebook_set_tab_pos(GTK_NOTEBOOK(nb), GTK_POS_TOP);

    GtkWidget *tabs[8];
    tabs[0] = build_category_tab(state, win);
    tabs[1] = build_person_tab(state, win);
    tabs[2] = build_priority_tab(state, win);
    tabs[3] = build_gift_tab(state, win);
    tabs[4] = build_parking_tab(state, win);
    tabs[5] = build_seating_tab(state, win);
    tabs[6] = build_schedule_tab(state, win);
    tabs[7] = build_stats_tab(state);

    const char *tab_labels[] = {
        "Categories", "Persons", "Priority", "Gifts",
        "Parking", "Seating", "Schedule", "Dashboard"
    };
    for (int i = 0; i < 8; i++) {
        GtkWidget *tab_lbl = gtk_label_new(tab_labels[i]);
        gtk_notebook_append_page(GTK_NOTEBOOK(nb), tabs[i], tab_lbl);
    }

    gtk_container_add(GTK_CONTAINER(win), nb);
    gtk_widget_show_all(win);

    /* Ensure totals are correct even if gifts file existed */
    if (g_gift_ctx) gift_refresh(g_gift_ctx);
    if (g_park_ctx) parking_refresh_tree(g_park_ctx);
    if (g_seat_ctx) seating_refresh_tree(g_seat_ctx);
    if (g_sched_ctx) schedule_refresh_tree(g_sched_ctx);
    if (g_stats_ctx) stats_refresh(g_stats_ctx);

    gtk_main();
}
```

Ln 2009, Col 1 | Spaces: 4 | UTF-8 | CRLF | { } C | Windsurf: Login | Win32 | Continue (NE)

6. Brainstorming Three Additional Modules

After completing the five required modules, the team met to identify real-world wedding management problems that WIGMS does not yet address. Three additional modules were proposed, designed, and implemented to extend the system into a complete wedding operations platform.

6.1 Brainstorming Session Summary

Problem Identified	Real-World Scenario	Proposed Module
Vehicles arriving at the venue have no organised system for parking	At a large Yaoundé wedding, 80+ cars and motorcycles arrive simultaneously with no coordination, causing congestion and VIPs unable to find spots	Module 6 Parking Management
Guests do not know where to sit in the dining hall	Without a seating plan, guests sit randomly, VIPs end up at general tables, and the caterer does not know how many meals of each type to prepare	Module 7 Meal & Table Seating
The MC and organisers have no tool to track the wedding day timeline	Events run late with no visibility of what is next, speeches overrun, the first dance is forgotten, and the photographer misses key moments	Module 8 Event Timeline & Schedule

1. Module 6 : Parking Management

7.1 Concept

Module 6 manages the wedding venue parking lot. The lot is divided into four zones: Zone A (VIP Reserved), Zone B (Standard Cars), Zone C (Motorcycles), and Zone D (Buses and Vans). Each zone has a fixed capacity. Each spot has a status that follows a state machine: AVAILABLE → OCCUPIED or RESERVED → AVAILABLE on departure.

2. Module 7 : Meal & Table Seating Management

8.1 Concept

Module 7 manages the dining hall seating arrangement. The hall contains tables of different types VIP head table, family tables, friends tables, children's table, and staff table. Each table has a fixed number of seats. Guests are assigned to specific seats with their meal preference and dietary restriction. The module also produces a MealSummary for the caterer.

3. Module 8: Event Timeline & Schedule Management

9.1 Concept

Module 8 manages the complete wedding day timeline using a doubly linked list of EventNode records. Unlike the singly linked list in Module 2, each node has both a *next and a *prev pointer, enabling bidirectional traversal and O(1) deletion. Events are automatically inserted in chronological order by scheduled_start time, maintaining a sorted list without a separate sort step.

9.2 Doubly Linked List vs Singly Linked List

Feature	Module 2 (Singly)	Module 8 (Doubly)
Pointers per node	*next only	*prev AND *next
Traversal direction	Forward only	Both forward and backward
Deletion	Requires previous pointer O(n) scan	Direct: node->prev->next = node->next O(1)
Insertion	O(1) at head only	O(n) to find position, but sorted automatically
Use case	Guest categories (order not critical)	Wedding timeline (chronological order essential)

10. Integration of All Eight Modules

10.1 Updated Project Structure

```
/WIGMS
 /include
 types.h ← shared structs (all modules depend on this)
 person.h ← Module 1
 category.h ← Module 2
 priority.h ← Module 3
 gift.h ← Module 4
 ui_gtk.h ← Module 5 (GTK interface + AppState)
 parking.h ← Module 6 (NEW)
 seating.h ← Module 7 (NEW)
 schedule.h ← Module 8 (NEW)
 /src
 main.c ← entry point initialises all modules
 person.c ← Module 1
 category.c ← Module 2
 priority.c ← Module 3
 gift.c ← Module 4
 ui_gtk.c ← Module 5
 parking.c ← Module 6
 seating.c ← Module 7
 schedule.c ← Module 8
 /data
 gifts.dat ← persisted gift array
 parking.dat ← persisted parking lot
 seating.dat ← persisted dining hall
 schedule.dat ← persisted event snapshots
 /reports
 Report1.docx / Report1.pdf
 Report2.docx / Report2.pdf
 Makefile
 README.md
```

10.4 Module Dependency Map

Module	Depends On	Provides To
types.h	Nothing	All modules
Module 1 person.c	types.h	category.c, seating.c, parking.c, ui_gtk.c
Module 2 category.c	types.h, person.h	priority.c, ui_gtk.c
Module 3 priority.c	types.h, category.h	ui_gtk.c
Module 4 gift.c	types.h, person.h	ui_gtk.c
Module 5 ui_gtk.c	All of the above + gtk/gtk.h	main.c (entry point calls ui_gtk_run)
Module 6 parking.c	types.h, time.h	ui_gtk.c (new Parking tab)
Module 7 seating.c	types.h, person.h	ui_gtk.c (new Seating tab)
Module 8 schedule.c	types.h, time.h	ui_gtk.c (new Schedule tab)

11. Challenges Encountered & Solutions Applied

Module	Challenge	Root Cause	Solution Applied
3 Priority	Merge sort returned wrong head on single-node list	Base case condition did not handle head->next == NULL	Added explicit check: if (!head || !head->next) return head
4 Gift	gift_id_counter reset to 1 after loading from file	Counter not updated after fread new gifts got duplicate IDs	Added: gift_id_counter = gifts[read-1].gift_id + 1 after load
5 GTK	Segfault when clicking sort button before any categories added	Callback accessed state->cat_head which was NULL	Added NULL guard at the top of every callback before dereferencing
5 GTK	CSS not applying buttons stayed default gray	ui_apply_css() was called after widget creation	Moved ui_apply_css() to the first line of ui_gtk_run(), before any gtk_widget_new() calls
6 Parking	time() returning same value for entry and exit in fast tests	Fast test completed in under 1 second difftime returned 0	Added sleep(1) in unit test; production code works correctly
7 Seating	seating_move_guest() lost RSVP status after move	seating_assign_guest() always resets rsvp to PENDING	Saved rsvp value before remove, restored it after assign
8 Schedule	fwrite of EventNode saved garbage pointer values	*prev and *next contain memory addresses, not data	Defined EventSnapshot without pointer fields; save/load use only snapshots

Module	Challenge	Root Cause	Solution Applied
8 Schedule	Doubly linked deletion corrupted list when removing tail	tail = e->prev was missing when removing last node	Added explicit tail update: if (!e->next) sched->tail = e->prev

12.2 Module Tests

Module	Test Performed	Expected Result	Status
3 Priority	Sort 6-category list by guest count using Merge Sort	List is in descending guest count order in one pass	To verify
4 Gift	Save 5 gifts, restart program, verify reload	5 gifts appear on startup with correct IDs	To verify
5 GTK	Click Add Category, fill form, verify TreeView updates	New category row appears instantly in the table	To verify
5 GTK	Close window, verify gifts auto-saved	gifts.dat file exists and is non-empty after exit	To verify

14. Conclusion

This second weekly report has documented the complete implementation of Modules 3 through 8 of the WIGMS project. Module 3 introduced the DPR (Divide-Process-Recombine) paradigm through a Merge Sort implementation on a linked list, demonstrating O(n log n) sorting without random access. Module 4 introduced binary file I/O with fwrite and fread, enabling gift data to persist between program sessions. Module 5 delivered the complete GTK+ 3.0 graphical user interface with 5 tabs, CSS theming, the signal-callback event model, and the AppState pattern for sharing data across the widget tree.

Three additional modules were designed through a team brainstorming session to address real wedding operations problems not covered by the original specification. Module 6 (Parking) introduced time.h timestamps and a spot status state machine. Module 7 (Seating) demonstrated nested struct arrays, multi-level search, and struct return by value for the catering order. Module 8 (Schedule) introduced the most advanced data structure in the project the doubly linked list with sorted chronological insertion, O(1) deletion, and an Event Snapshot serialisation strategy to safely write pointer-containing structs to disk.

All eight modules compile cleanly together, are committed to the GitHub repository, and are ready for integration testing and final submission. The next steps are to complete all pending GTK tab implementations for Modules 6-8, conduct the full system integration test documented in Section 12, and prepare the final technical report.